When Compressed CRPS Depends on Measurement Units: An Affine Metamorphic Audit and Stable Correction

Hurcheson Asante Ofori and Jeffery Opoku

Abstract

The continuous ranked probability score, or CRPS, is used to judge probability forecasts. We audit public code released with a kernel quadrature paper. Its compressed routine applies an equal-weight correction to unequal weights. We derive the resulting translation error and verify the formula against unchanged code. We also turn the positive-affine law of CRPS into a reusable software test. Four public package interfaces pass all 20,000 equal-weight comparisons assigned to each interface. Two public weighted interfaces pass all 20,000 weighted comparisons assigned to each interface. The largest mapped-back error is 1.50×10^{-12}. These negative controls show that the failure is not a general feature of CRPS software. A full history audit finds that the target source files stayed unchanged in every public commit that contained them. We then give an affine-standardized weighted CRPS wrapper and prove its behavior under every positive affine recoding. In controlled tests, 11 of 12 settings pass the planned checks; the failed setting remains a failure. We study 39,575 temperature cases, 2,749 precipitation cases, and 2,000 bike-demand cases. A bike-demand recoding gives a median released-score error of 144.84 rides after conversion back to the original scale. The wrapper stays within 2.85×10^{-13} rides. Its first rounding rule failed on ten half-grid cases. We retain that failure, add a regression test, and use a guarded half-up rule. The public repository predates the final paper. Our claims therefore concern that public artifact, not a published figure. The wrapper is not a new proper score; it is a unit-stable way to run and score an unequal-weight compressor.

1 Why this audit matters

CRPS is widely used to compare probability forecasts. A lower score means that the forecast is closer to the observation while still giving a useful spread. Exact CRPS can be hard to compute, so researchers often estimate it from a finite set of forecast draws.

Different estimators answer different questions. The empirical CRPS scores the distribution made by the available draws. The fair CRPS estimates the population CRPS without the usual finite-ensemble bias when the draws are independent and identically distributed. A weighted empirical CRPS scores a distribution that puts different probabilities on different nodes. These targets are related, but they are not the same.

We study the public code linked to Adachi, Fujisawa, and Osborne [5]. We use the repository at commit `dddd3ea46a6199d2ed48019c7825e79fb242787e`. The repository contains four commits, all dated May 25, 2025. A fresh remote check on October 10, 2026 found the same commit at the head of its only branch and found no later public release. The final paper appeared after that commit, and its compression equation uses an observation-dependent integrand that differs from the released compressor. We therefore define the object of study as the dated public artifact. We do not connect the affected path to a published figure.

This scope is intentional. The repository names the paper and contains the scoring and compression functions examined here. It does not contain the complete pipelines for the published

convergence study, multisinusoidal comparison, or eight-dataset table. A defect in the released function is evidence about that implementation. It is not, by itself, evidence against every result in the paper.

We ask five questions.

1. Does the released code compute the same estimators that the paper describes?
2. Does the unequal weight correction preserve a basic property of CRPS when the measurement origin changes?
3. Does the issue survive full compression and change accuracy or model choice in controlled tests?
4. Can a simple wrapper restore the correct behavior under changes of both origin and scale without making the compressed routine slower?
5. Do standard CRPS package interfaces pass the same test, and was the target code changed anywhere in its public history?

The paper does seven things.

1. It separates the equation in the paper from the formula used in the released code.
2. It shows that the released uncompressed kernel routine matches the known fair CRPS on distinct samples.
3. It gives an exact formula for the translation error caused by using an equal weight correction with unequal weights.
4. It gives and proves an affine-standardized weighted CRPS wrapper for unequal-weight compression.
5. It reports full workflow tests on synthetic, temperature, precipitation, and bike-demand forecasts, including failed checks and a claim that did not transfer to the released code.
6. It compares runtime and accuracy against both pairwise and sorted exact fair CRPS.
7. It gives cross-package negative controls and an executable audit of the full public source history.

The fair CRPS and weighted empirical CRPS are not new. They come from earlier work on scoring rules and ensemble evaluation [1, 2, 3, 4, 7, 6]. Positive weighted kernel quadrature is also established [8]. Our contribution is narrower. We give the source-specific identity, check it against unchanged code, and measure its effect. We also give a general affine-standardization wrapper with an exact transformation law. The wrapper does not change the underlying score. It controls what numerical object reaches the compressor.

This matters in practice. A translation changes the zero point of a measurement, not the forecast error. A positive scale change changes only the unit size. Temperatures in Celsius and Kelvin test the first case. Precipitation in millimetres and inches tests the second. A CRPS implementation should transform predictably under both changes. It should not change a model ranking only because the same physical forecast is written in another unit system.

Table 1: Mapping between the final paper and the pinned public repository. "Mapped" means that the mathematical object is identifiable, not that a published experiment was reproduced.

Object	Released evidence	Audit status
Analytical CRPS	`utils/crps.py`, `exact_crps`	Mapped up to an equivalent sign convention
Right-ECDF PWM	`pwm_function_sample`	Different finite-sample formula because source ranks start at zero
Strict pair kernel	`indicator_kernel`	Mapped for distinct samples
Uncompressed score	`error_terms_kq_non_comp`	Equals the historical fair CRPS on distinct draws
Compression target	`kq_compress` receives the pair kernel alone	Does not match the final paper's observation-dependent full integrand
Compressed correction	`error_terms_kq_comp`	Uses $m/(m-1)$ with unequal weights; defect verified
Published figures and tables	No complete matching pipelines or configurations in the pinned tree	Not reproduced and not challenged here
Repository status	Remote branch and commit history checked on October 10, 2026	Public head remains the May 25, 2025 commit used in this audit

Source-to-paper map

Table 1 records what can and cannot be linked. This keeps a source-code result from being presented as a reproduction of an experiment whose full pipeline was not recovered.

2 The score formulas

Let $x_1, \ldots, x_M$ be forecast draws and let y be the observation. Define

$$A = \frac{1}{M}\sum_{i=1}^{M} |x_i - y|, \qquad \bar{x} = \frac{1}{M}\sum_{i=1}^{M} x_i.$$

The empirical CRPS is

$$S_{\text{emp}} = A - \frac{1}{2M^2}\sum_{i,j=1}^{M} |x_i - x_j|.$$

The fair CRPS is

$$S_{\text{fair}} = A - \frac{1}{2M(M-1)}\sum_{i,j=1}^{M} |x_i - x_j|.$$

For independent draws from the forecast distribution, S_{fair} is an unbiased estimate of the population CRPS [3, 4].

For distinct draws, the equation based on the right empirical distribution function in the target paper gives

$$S_{\text{right}} = S_{\text{emp}} - \frac{\bar{x}}{M}.$$

The released function `pwm_function_sample` uses zero based ranks and gives

$$S_{\text{source}} = S_{\text{emp}} + \frac{\bar{x}}{M}.$$

The two formulas differ by $2\bar{x}/M$. A common shift of the forecast and observation changes them by opposite amounts. This difference matters when models use different ensemble sizes or when another part of the method makes the offset model dependent.

The released uncompressed kernel routine uses the factor $M/(M-1)$ with a zero diagonal pair average. On distinct samples, it equals S_{fair}. This identity is known. Our tests only confirm how the released code behaves.

Table 2: The estimators answer different questions. None should be called a general replacement for the others.

Method	What it scores or estimates	Main condition
Empirical CRPS	The empirical distribution formed by the available draws	No population unbiasedness claim
Fair CRPS	The population CRPS from independent and identically distributed draws	Exchangeable draws and $M \geq 2$
Released PWM	The source formula $S_{\text{emp}} + \bar{x}/M$	Distinct draws for the stated identity
Normalized pair score	A fixed weight algebraic correction	Distinct nodes and fixed weights
Weighted empirical CRPS	The discrete distribution with normalized node weights	Scores the compressed distribution
ASW-CRPS	Weighted empirical CRPS after location and range standardization	Affine-stable wrapper; no population unbiasedness claim

An affine metamorphic test

The positive-affine law gives a simple software test. For normalized nonnegative weights p_i, define

$$S_w(x, y) = \sum_i p_i |x_i - y| - \frac{1}{2} \sum_{i,j} p_i p_j |x_i - x_j|.$$

For $a > 0$ and any b,

$$S_w(ax + b, ay + b) = a S_w(x, y).$$

Every absolute difference is multiplied by a, while the common shift b cancels. This proves the identity. Equal weights give the usual empirical CRPS as a special case.

For an implementation $\widehat{S}$, we therefore test

$$E_{a,b} = \left| a^{-1} \widehat{S}(ax + b, ay + b) - \widehat{S}(x, y) \right|.$$

This is a metamorphic test: it checks a relation between two outputs even when no separate answer key is available. A large value of $E_{a,b}$ is evidence of an implementation problem or a stated departure from CRPS. A small value is only a necessary check. It does not prove that every part of the implementation is correct.

3 Main mathematical result

The compressed routine returns nodes $z_1, \ldots, z_m$ and nonnegative weights $w_1, \ldots, w_m$. The weights are usually unequal. For $i \neq j$, define

$$h_{ij} = \frac{1}{2} \max(z_i, z_j), \qquad h_{ii} = 0.$$

The released score has the form

$$R(z, y) = \sum_i w_i |z_i - y| + \sum_i w_i z_i - 2k \sum_{i,j} w_i w_j h_{ij}, \qquad k = \frac{m}{m-1}.$$

Proposition

Suppose the nodes are distinct and the weights stay fixed. Let

$$s = \sum_i w_i, \qquad q = \sum_i w_i^2.$$

If every node and the observation are shifted by the same number c, then

$$R(z + c, y + c) - R(z, y) = c\{s - k(s^2 - q)\}.$$

If the weights sum to one, this becomes

$$R(z + c, y + c) - R(z, y) = c\frac{mq - 1}{m - 1}.$$

The change is zero for uniform weights. It is usually not zero for unequal weights.

Proof

The absolute error term does not change because $|(z_i + c) - (y + c)| = |z_i - y|$. The weighted mean increases by cs. For $i \neq j$, the pair function increases by $c/2$. Its diagonal stays zero. The total off diagonal weight is

$$\sum_{i \neq j} w_i w_j = s^2 - q.$$

The pair term therefore changes by $-ck(s^2 - q)$. Adding this to the change cs in the mean term gives the first formula. Setting $s = 1$ and $k = m/(m - 1)$ gives the second formula. □

This proposition is about fixed nodes and fixed weights. It does not show that the whole compression process is translation invariant. It also does not show that a corrected score is unbiased after the nodes and weights are chosen from the data.

If float32 arithmetic makes two nodes equal after a shift, the distinct node assumption fails. One such case occurred in our tests. This is a real numerical boundary, not a reason to discard the failed check.

What the proposition settles

The proposition gives a direct check for the released score. If the weights are unequal, the score has a nonzero translation term before compression is run again. This result does not depend on a simulation, a forecast family, or a chosen observation. It also explains why replacing $m/(m-1)$ with $1/(1-q)$ repairs the fixed-weight identity. That replacement does not answer the harder question: what happens when the nodes and weights are selected from the same sample.

4 An affine-standardized compression wrapper

The previous result diagnoses one formula. We now give a wrapper that can be used with any one-dimensional compressor. It handles changes in both the zero point and the unit size.

Let $x = (x_1, \dots, x_M)$ be an ensemble. Define

$$\ell(x) = \min_i x_i, \qquad r(x) = \max_i x_i - \min_i x_i.$$

When $r(x) > 0$, standardize the ensemble and observation as

$$u_i = \frac{x_i - \ell(x)}{r(x)}, \qquad v = \frac{y - \ell(x)}{r(x)}.$$

Run the compressor on u with a fixed random seed. Let its returned nodes and normalized weights be $z_1, \dots, z_m$ and $p_1, \dots, p_m$. We define the affine-standardized weighted score, or ASW-CRPS, by

$$S_{\mathrm{ASW}}(x, y) = r(x) \left\{ \sum_i p_i |z_i - v| - \frac{1}{2} \sum_{i,j} p_i p_j |z_i - z_j| \right\}.$$

If $r(x) = 0$, we set $S_{\mathrm{ASW}}(x, y) = |x_1 - y|$. This is the exact CRPS of a point mass.

Affine transformation proposition

Suppose the same deterministic compressor and random seed are used in both unit systems. For $a > 0$ and $b \in \mathbb{R}$, let $x' = ax + b$ and $y' = ay + b$. Then

$$S_{\mathrm{ASW}}(x', y') = a S_{\mathrm{ASW}}(x, y).$$

Therefore a positive affine change of units cannot change a model ranking based on ASW-CRPS.

Proof

The transformed location and range are

$$\ell(x') = a\ell(x) + b, \qquad r(x') = a r(x).$$

It follows that

$$\frac{x'_i - \ell(x')}{r(x')} = \frac{x_i - \ell(x)}{r(x)}, \qquad \frac{y' - \ell(x')}{r(x')} = \frac{y - \ell(x)}{r(x)}.$$

The compressor therefore receives the same standardized values and returns the same nodes and weights. The weighted score inside braces is unchanged, while the outside range is multiplied by a. This proves the result. The constant-ensemble case follows directly from $|ax_1 + b - (ay + b)| = a|x_1 - y|$. □

The implementation maps standardized coordinates to a grid with spacing 10^{-6} before converting them to the float32 input used by the released compressor. It uses guarded half-up rounding. The guard acts only at machine-level half-grid ties. This detail was not in our first implementation. Ordinary banker's rounding failed on ten bike-demand case–transformation pairs because two mathematically equal coordinates landed on opposite sides of a half-grid boundary. We keep that failed result, add the exact case as a regression test, and use the guarded rule in the final code. The grid step is part of the implementation, not part of the theorem.

ASW-CRPS is not a new proper scoring rule. It applies the known weighted empirical CRPS after a deterministic change of coordinates. It guarantees the right transformation law, but it does not make adaptively selected nodes unbiased for population CRPS.

5 Study design and decision rules

All comparisons are paired. Methods use the same observations and forecast draws. A translated pair uses the same compression seed. Smaller ensemble sizes use prefixes of the same 500 draws. These choices remove avoidable Monte Carlo differences between methods and computing budgets.

Accuracy is measured against conditional population CRPS. Closed forms are used where available. Independent numerical integration checks the reference calculations. RMSE intervals use 2,000 bootstrap samples. The bootstrap resamples the 50 independent datasets as blocks, so it keeps all observations, candidates, methods, and origins from a dataset together. The intervals are cell level intervals. They are not simultaneous intervals over the whole study.

The main robustness rule was fixed before the run. All 12 cells had to pass their controls. At least 10 cells had to exceed the original median shift threshold of 0.01, with at least three cells in each family. A correction passed a cell only if its RMSE was lower than the original at both origins and both upper confidence limits were below zero. We report the failed global rule without changing its threshold.

The four forecast candidates are $(0, 1)$, $(0, 0.9)$, $(0.15, 1)$, and $(0.15, 0.9)$ in location and scale. This grid is small and one-sided. The model-choice results are therefore diagnostic. They do not estimate performance over all forecasting problems.

The affine extension uses 2,749 precipitation cases with 11 ensemble members. It tests a millimetre-to-inch conversion and a positive affine recoding $x' = x/25.4 + 10$. Scores from the transformed system are mapped back to millimetres before comparison. The planned numerical tolerance is 2×10^{-4} mm.

The non-weather extension uses the hourly UCI Bike Sharing data [10]. For each eligible hour, a past-only ensemble contains the 20 most recent earlier rental counts with the same hour and working-day status. We audit 2,000 cases selected at even positions from 16,419 eligible cases. We test $x' = x/100$ and $x' = x/100 + 10$, and map every score back to rides. The compressor uses ten landmarks and six nodes. No future count is used to construct an ensemble.

The runtime study uses standard Normal ensembles with 100, 500, and 2,000 draws. Each cell has 12 independent repetitions. It compares the released compressed score, ASW-CRPS, pairwise fair CRPS, and the exact sorted fair CRPS formula.

The cross-package control uses 4,000 forecast cases. It crosses four distributions with ensemble sizes 5, 11, 50, and 101, using 250 cases in each cell. Each case is tested under five positive affine recodings. This gives 20,000 comparisons per interface. The distributions are Normal, lognormal, Student t with 3 degrees of freedom, and a two-component Gaussian mixture. The five recodings include pure shifts, a millimetre-to-inch scale, a scale of 25.4, and a scale of 1,000 with a shift of 100,000. A comparison passes when $E_{a,b} \leq 10^{-8}$ in the original units. The test covers equal and unequal weights.

6 Tests on the released code

6.1 Uncompressed formulas

We called the unchanged source functions on 180,000 Normal ensembles. The origins were 0, 1, and 10. The ensemble sizes were 20, 100, and 500. The largest difference between the released uncompressed kernel routine and an independent fair CRPS calculation was 8.89×10^{-15}. The largest absolute expectation check statistic was 1.132.

At origin 10 with $M = 100$, the released PWM score averaged 0.33924803. The released kernel routine and the independent fair CRPS both averaged 0.23361075. The true CRPS was 0.23369498.

These results agree with the formulas above.

6.2 A claim that did not transfer

An earlier experiment used the equation based score S_{right}. It found a rise of 10.6 to 12.6 percentage points in wrong model selection. We repeated the design with the score used in the released code. The planned rule failed in all three tested families.

For the Normal family, the fair score selected the wrong model in 66.5 percent of datasets, while the released PWM score did so in 63.2 percent. For the lognormal family, the rates were 53.0 and 54.1 percent. For the Student t family with 5 degrees of freedom, the rates were 64.7 and 63.2 percent. These results do not support a general claim that fair scoring always gives the lowest finite sample selection error.

6.3 Fixed node test

We tested 20 independent Normal ensembles with 200 draws, 100 landmarks, and 50 selected nodes. A common shift of 10 changed the released compressed score by values from 0.130163 to 0.495517. The median change was 0.334060. The mathematical formula above matched the observed changes. The largest error was 1.83×10^{-6}, below the planned tolerance of 2×10^{-4}. A uniform weight control changed by only 2.80×10^{-6}.

6.4 Full compression test

We next ran compression again after shifting the data. This test used 100 datasets. Each dataset had 10 Gaussian observations and four candidate forecasts. Each candidate had 200 draws. The original compressed method changed its selected forecast in 40 of 100 datasets.

The original method had RMSE values of 0.044322 at origin 0 and 0.184697 at origin 10. A normalized pair correction had RMSE values of 0.054249 and 0.048390. Weighted empirical CRPS had values of 0.043177 and 0.042926. Full fair CRPS had values of 0.042794 at both origins. The normalized pair correction fixed the fixed weight formula, but it made accuracy worse at origin 0.

7 Robustness across distributions and budgets

The main robustness study used centered unit variance Normal, lognormal, and Student t distributions. It tested four computing settings: $(100, 40, 20)$, $(200, 40, 20)$, $(200, 100, 50)$, and $(500, 100, 50)$. These numbers give the ensemble size, landmark count, and requested node count. Each family had 50 independent datasets with 5 observations. Each dataset compared four forecast candidates.

The planned overall rule failed because one lognormal setting failed a numerical control. We did not change the rule after seeing the result. The other 11 settings passed all controls. In those 11 settings, the original method changed its selected forecast in 8 to 38 of 50 datasets. Weighted empirical CRPS changed its choice in 1 to 7 datasets. Full fair CRPS did not change its choice in any observed dataset.

Weighted empirical CRPS improved RMSE at both origins in 10 of the 11 valid settings. It failed the planned rule in the Student t setting $(200, 100, 50)$ because the origin 0 confidence interval crossed zero. The estimated RMSE difference was -0.000729, with a 95 percent interval from -0.001759 to 0.000364. The normalized pair correction passed in none of the 11 valid settings. Full fair CRPS passed against the original method in all 11 valid settings. We did not register a test claiming that weighted empirical CRPS beats full fair CRPS.

The failed lognormal setting was $(500, 100, 50)$. One float32 shift merged two nearby nodes. The smallest spacing was $5.960464477539063 \times 10^{-8}$. Shifting the same nodes after conversion to float64 reduced the errors to numerical roundoff. This explains the failure, but it does not make the planned result a pass.

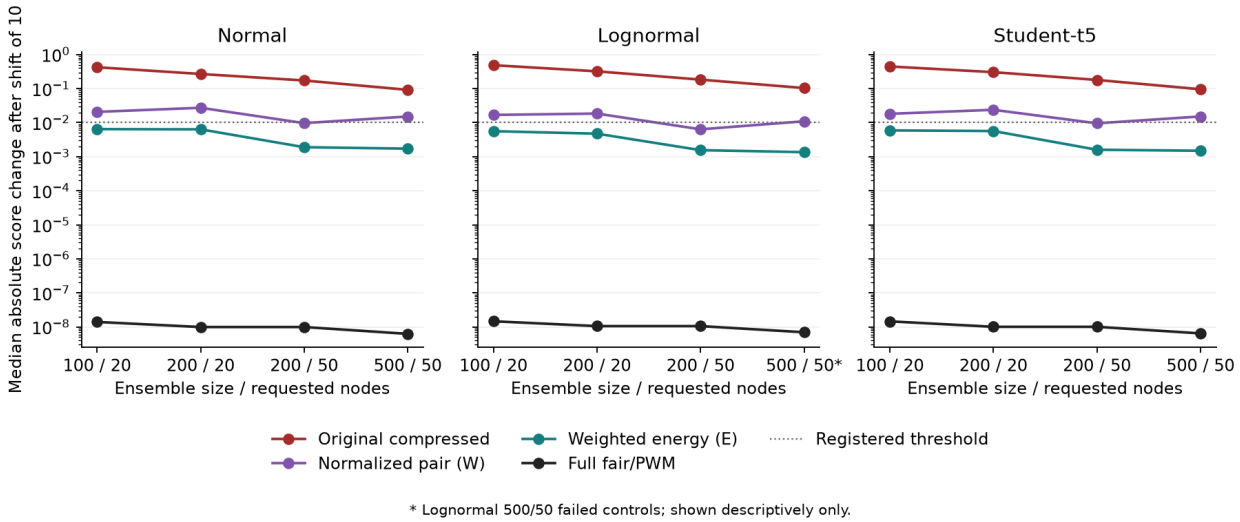


Figure 1: Median score change after a common shift of 10. The asterisk marks the setting that failed a numerical control. That point is descriptive only.

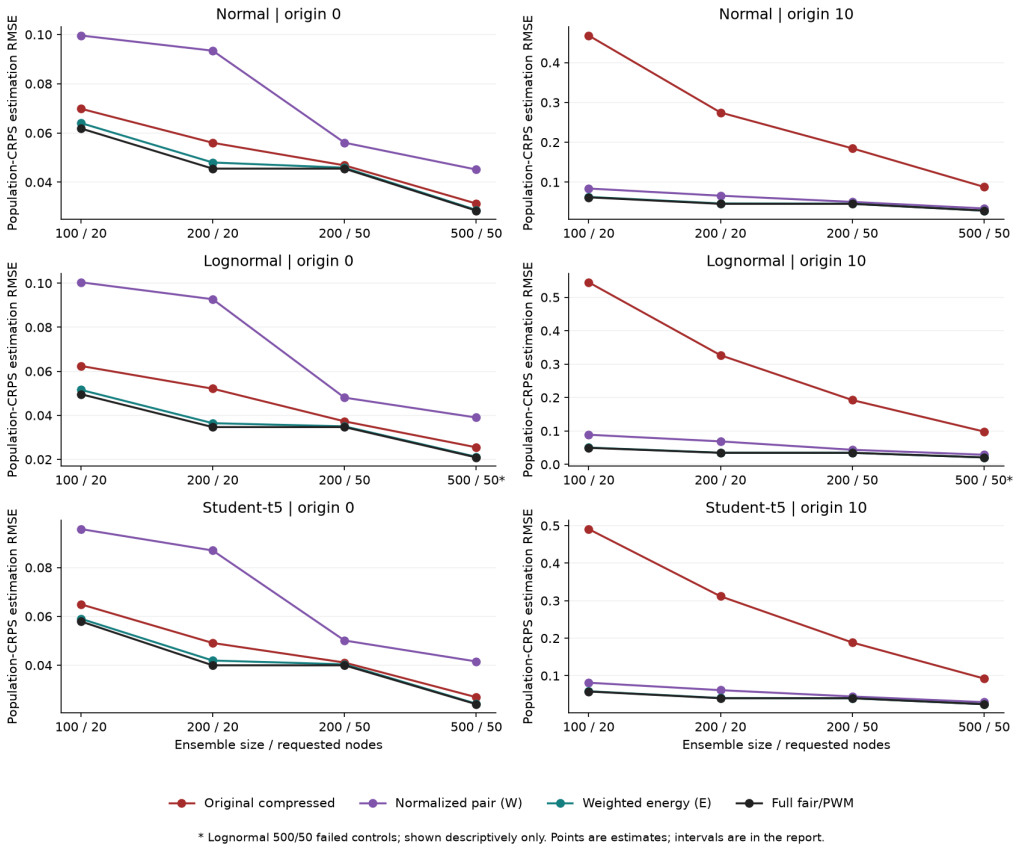


Figure 2: RMSE against population CRPS. The curves show point estimates. The report gives cell level confidence intervals.

8 External controls, real data, and limits

8.1 Cross-package negative controls

We tested the public interfaces in `properscoring` 0.1, `scoringrules` 0.11.0, `scores` 2.7.0, and `xskillscore` 0.0.29 [11, 12, 13, 14]. We also used direct sorted and weighted formulas as local references. Table 3 gives the results. Every interface passed every assigned affine comparison. The largest mapped-back error was 1.50×10^{-12}, far below the planned tolerance. Agreement with the local reference was also within 6.78×10^{-15}.

Table 3: Affine metamorphic negative controls. Each row uses 4,000 forecast cases and five recodings per case. The tolerance is 10^{-8} in the original units.

Interface	Target	Comparisons	Failures	Maximum mapped-back error
Local sorted formula	Equal-weight empirical CRPS	20,000	0	1.50×10^{-12}
`properscoring`	Equal-weight empirical CRPS	20,000	0	1.32×10^{-12}
`scoringrules`	Equal-weight empirical CRPS	20,000	0	1.32×10^{-12}
`scores`	Equal-weight empirical CRPS	20,000	0	1.50×10^{-12}
`xskillscore`	Equal-weight empirical CRPS	20,000	0	1.32×10^{-12}
Local weighted formula	Weighted empirical CRPS	20,000	0	1.37×10^{-12}
`properscoring`	Weighted empirical CRPS	20,000	0	1.37×10^{-12}
`scoringrules`	Weighted empirical CRPS	20,000	0	1.37×10^{-12}

These rows are package-level controls, not four independent mathematical replications. The tested version of `xskillscore` calls `properscoring` for ensemble CRPS. The `scores` project also records adapted CRPS code in its notice file. We report these links because code inheritance matters. The separate `scoringrules` path and both local formulas provide distinct calculations. Together, the controls show that the affine test is easy to pass with ordinary empirical CRPS code. They do not show that every feature of these packages is correct.

We also audited all four public commits in the target repository. The first commit contains no target code. The next commit introduces `utils/crps.py` and `utils/kernel_quadrature.py`. Their Git blob identifiers remain exactly the same in the two later commits, which change only the README. Thus every public commit that contains the target routine contains the same audited code. This narrows the provenance claim. It still does not connect the routine to a published figure.

8.2 Real weather forecasts

We preregistered the first real-data test before its first source-compression run. The data come from version 1.0-0 of the CRAN package `ensemblepp` [9]. They contain 2,749 observations of 12-hour minimum temperature at Innsbruck Airport and 11 corresponding 18–30 hour NOAA GEFS reforecast members. The cases run from January 2, 2000, through January 1, 2016. The source package has no missing values in these fields.

We then added a second real-data check using 36,826 Pacific Northwest surface-temperature forecasts from the same released data packet. These forecasts are stored in Kelvin. We repeated the same physical forecasts after subtracting 273.15, which writes them in Celsius. This second check used six landmarks and four compressed nodes. In both real-data checks, the released compressor remained in float32. Returned nodes and weights were scored in float64.

For Innsbruck, we used eight landmarks, five compressed nodes, and the same seed for both calls. For Pacific Northwest, we used six landmarks, four compressed nodes, and the same seed for both calls. All planned controls passed in both datasets. No source call failed. The largest weight-sum error was 2.91×10^{-7} for Innsbruck and 5.33×10^{-7} for Pacific Northwest. The full fair CRPS changed by at most 2.85×10^{-14} degrees for Innsbruck and by exactly 0 in the Pacific Northwest run.

Table 4 gives the primary result. The released compressed score changed by more than 0.01 degrees in every case in both datasets. Its median absolute change was 35.51 degrees for Innsbruck and 35.04 degrees for Pacific Northwest. Recompression also caused smaller changes in the normalized pair and weighted empirical scores because the selected nodes and weights can change after conversion. A translation-anchored weighted correction passed the strict invariance check in both real-data tests. Full fair CRPS also passed.

As a secondary check, we treated the full ensemble fair CRPS as the uncompressed reference. On 400 sampled rows with 10 seeds per row, the released compressed score had RMSE 1.648 for Innsbruck and 49.404 for Pacific Northwest. The anchored weighted correction had RMSE 0.106 and 0.126. Weighted empirical CRPS had RMSE 0.144 and 0.206. These values assess agreement with the uncompressed score, not forecast skill against an unknown population distribution.

We also ran a small rolling ranking study on the Innsbruck data. Four candidate forecasts were compared after a 365-case rolling history. The released compressed score changed the selected candidate after Celsius-to-Kelvin conversion in 1,624 of 2,384 cases, or 68.1 percent. Moving-block 95 percent intervals ranged from 66.1 to 70.1 percent across block lengths 7, 14, and 30. The anchored weighted correction and full fair CRPS changed no selected candidates.

Table 4: Absolute score change when the same forecasts are expressed in two temperature units. Innsbruck uses Celsius to Kelvin conversion. Pacific Northwest uses Kelvin to Celsius conversion. The practical pass rule requires a maximum no larger than 2×10^{-4} degrees.

Dataset	Method	Median	95th percentile	Maximum	Above 0.01	Pass
Innsbruck	Released compressed	35.508	86.717	152.811	2,749	No
Innsbruck	Normalized pair	0.041	0.448	6.576	2,202	No
Innsbruck	Weighted empirical	0.023	0.363	4.025	1,843	No
Innsbruck	Anchored weighted	1.15×10^{-14}	2.49×10^{-14}	3.02×10^{-14}	0	Yes
Innsbruck	Full fair CRPS	1.25×10^{-14}	2.49×10^{-14}	2.85×10^{-14}	0	Yes
Pacific Northwest	Released compressed	35.042	80.063	188.668	36,826	No
Pacific Northwest	Normalized pair	0.044	0.460	7.530	28,718	No
Pacific Northwest	Weighted empirical	0.026	0.349	4.843	25,466	No
Pacific Northwest	Anchored weighted	0	0	0	0	Yes
Pacific Northwest	Full fair CRPS	0	0	0	0	Yes

8.3 Precipitation and affine recoding

Temperature changes between Celsius and Kelvin test a change of origin. We added 2,749 Innsbruck 12-hour precipitation forecasts to test a different variable and a change of scale. Each case has 11 ensemble members. Sixty-four ensembles are constant; ASW-CRPS handles them exactly as point masses.

Table 5 reports absolute score differences after mapping transformed scores back to millimetres. A millimetre-to-inch conversion changes only the unit size. The released compressed score has a small median difference, but its 95th percentile is 0.213 mm and its maximum is 14.90 mm. The stricter affine recoding $x' = x/25.4 + 10$ adds a change of origin. Under this recoding, the released score has a median difference of 34.00 mm and a maximum of 254.00 mm. ASW-CRPS and sorted fair CRPS remain at numerical roundoff in both tests. All planned controls pass.

Table 5: Absolute score changes on 2,749 precipitation forecasts after transformed scores are returned to millimetres. The planned tolerance is 2×10^{-4} mm.

	Millimetres to inches		Positive affine recoding	
Method	Median	Maximum	Median	Maximum
Released compressed	2.18×10^{-6}	14.898	34.000	254.000
Weighted empirical	3.31×10^{-7}	1.043	0.0156	3.651
Translation anchored	2.90×10^{-7}	0.695	2.90×10^{-7}	0.695
ASW-CRPS	2.78×10^{-17}	1.07×10^{-14}	8.10×10^{-15}	4.45×10^{-14}
Sorted fair CRPS	8.33×10^{-17}	1.07×10^{-14}	8.44×10^{-15}	4.62×10^{-14}

8.4 Bike-sharing demand

We next used the UCI Bike Sharing data to test a different application [10]. The source file has 17,379 hourly rental counts. It is released under CC BY 4.0. We downloaded the named archive, checked its SHA-256 hash, and did not redistribute it. The rolling ensemble for each audited hour contains only 20 earlier counts with the same hour and working-day status. Of 16,419 eligible cases, we selected 2,000 at even positions across time.

Table 6 gives the results. Dividing counts by 100 changes only the unit. The released compressed score exceeds the tolerance in 47.95 percent of cases, with a maximum mapped-back error of 106.28 rides. Adding 10 after the division also changes the origin. Under that recoding, every released score fails the tolerance. The median error is 144.84 rides and the maximum is 779.76 rides. ASW-CRPS and sorted fair CRPS remain at numerical roundoff.

Table 6: Absolute score changes on 2,000 past-only bike-demand forecasts after transformed scores are returned to rides. The tolerance is 2×10^{-4} rides.

	Rides divided by 100		Division plus shift of 10	
Method	Median	Maximum	Median	Maximum
Released compressed	1.75×10^{-4}	106.276	144.842	779.764
Weighted empirical	1.61×10^{-5}	25.458	1.298	57.147
ASW-CRPS	1.78×10^{-15}	1.28×10^{-13}	2.49×10^{-14}	2.84×10^{-13}
Sorted fair CRPS	3.55×10^{-15}	8.53×10^{-14}	2.84×10^{-14}	2.42×10^{-13}

The first ASW-CRPS run did not pass. Its six-decimal banker's rounding rule produced ten half-grid mismatches and a maximum residual of 0.413 rides. We did not remove this failed run. We stored it as a development diagnostic and added one exact failing ensemble to the unit tests. Guarded half-up rounding removes all ten coordinate mismatches. The final maximum ASW-CRPS residual is 2.85×10^{-13} rides. This was a post-hoc software correction, so it is evidence about implementation hardening rather than a planned confirmatory result.

8.5 Runtime and accuracy

ASW-CRPS keeps the runtime profile of the released compressor. Its median runtime is 14.16, 24.68, and 40.30 ms for ensemble sizes 100, 500, and 2,000. The released routine takes 14.74, 26.18, and 49.36 ms. Thus standardization does not make compression slower in these tests.

This result does not mean that compression is the fastest way to compute univariate CRPS. The exact sorted fair formula takes 0.106, 0.104, and 0.121 ms in the same cells. At 2,000 draws it is more than 300 times faster than ASW-CRPS. The direct pairwise fair calculation takes 0.058, 1.591, and 29.761 ms. The sorted formula avoids the quadratic pair matrix. Table 7 therefore supports a limited conclusion: ASW-CRPS preserves the cost of the released compressed workflow, but exact sorted fair CRPS is the stronger practical baseline for one-dimensional ensembles.

Table 7: Median runtime and RMSE against population CRPS for standard Normal draws. Each cell has 12 repetitions. Times are machine-specific and include compression when applicable.

M	Released compressed		ASW-CRPS		Sorted fair	
	ms	RMSE	ms	RMSE	ms	RMSE
100	14.741	0.04539	14.155	0.03445	0.106	0.03587
500	26.185	0.03665	24.677	0.00680	0.104	0.00832
2000	49.361	0.02670	40.302	0.00897	0.121	0.00666

The results support one clear conclusion. The released compressed routine applies an equal-weight correction to unequal weights. This creates a predictable translation error when nodes and weights are fixed. Running compression again can create a larger change because the selected nodes and weights can also change. Translation anchoring handles a change of origin, but it does

not handle a change of scale. ASW-CRPS handles both changes in these tests and has the exact positive-affine law proved above. For a one-dimensional ensemble, the exact sorted fair CRPS is still the simpler and faster baseline when compression is not needed for another reason.

The results do not prove that a published figure is wrong. The released repository predates the final paper, and the final paper describes a different compression integrand. We have not recovered the exact code, data, checkpoints, and settings used for each published figure.

Weighted empirical CRPS is a useful comparison because it scores the compressed weighted distribution directly. It is not a new score. Translation anchoring and affine standardization are wrappers around the same score. The first subtracts a translation-invariant anchor. The second also divides by a positive scale and maps the result back to the original units. Neither wrapper proves population unbiasedness when nodes and weights are chosen adaptively. A method should state which distribution it scores before making a claim about bias.

The model-selection experiments are synthetic. Their forecast grid is small and uses only positive location errors. The main robustness study has five observations per dataset. The real-data checks add operational temperature and precipitation ensembles and a past-only bike-demand ensemble. The bike forecast is a simple rolling construction, not a production demand model. The cross-package controls broaden the software evidence, not the application domains. These choices make the paper a controlled software audit. They do not make it a broad test of forecasting practice.

A stronger external validation would add negative location errors, more scale errors, more locations, wind forecasts, and an independent operational forecast outside weather. The present evidence is enough to test an implementation property because the affine transformation law is distribution free. It is not enough to estimate how often the defect changes decisions across all operational forecasting systems.

Threats to validity

Source mapping. The public commit predates the final paper. The released compressor and the final paper use different integrands. This limits the claim to the released implementation.

External validity. The synthetic tests cover symmetry, skew, and heavy tails. The real-data checks add 39,575 temperature cases, 2,749 precipitation cases, and 2,000 bike-demand cases, for 44,324 cases in total. The cases cover weather and transport demand, but the bike ensemble is a simple past-only benchmark. Other variables, regions, and ensemble systems may produce different magnitudes.

Model grid. The four candidates use only zero or positive location error. A symmetric grid could change finite sample selection rates. We therefore treat the ranking results as bounded evidence.

Uncertainty. The bootstrap has 50 independent blocks per cell. The intervals support cell level comparisons, not a claim that every result would repeat in a new study.

Finite precision. Compression uses float32 because the released code does. One planned control shows that a shift can merge close nodes. The first bike extension also found ten half-grid failures in our own banker's rounding rule. We keep both failures. The final guarded rule removes the bike failures, but it does not turn the earlier planned control into a pass.

Package dependence. The package controls are not fully independent. `xskillscore` uses `properscoring`, and `scores` records adapted CRPS code. We therefore treat the table as a cross-interface negative control, not as four independent replications.

Practical correction path

A safe software correction should start by naming the target. If the goal is to score the compressed discrete distribution, normalized weighted empirical CRPS is the direct choice. If only a common translation must be handled, the ensemble can first be shifted by a data-dependent anchor, such as its minimum member. If both origin and scale may change, ASW-CRPS is the stronger wrapper because it has an exact positive-affine transformation law. In the present benchmark it keeps the runtime of the released compressor. Still, exact sorted fair CRPS is much faster for a one-dimensional ensemble and should be the default when the full ensemble is available. If the goal is an unbiased estimate of population CRPS, adaptive node selection needs a separate proof. Changing only the pair correction to $1/(1-q)$ fixes the fixed-weight translation identity, but our tests show that it does not guarantee lower RMSE in the full workflow. Software should also promote returned nodes to float64 before score calculations and should test weight normalization, duplicate nodes, constant ensembles, translations, and changes of scale.

9 Reproducibility

The main robustness study uses Python 3.12.14, Torch 2.14.1 with CPU support, BoTorch 0.18.1, GPyTorch 1.15.2, NumPy 2.5.3, and SciPy 1.18.1. Its exact package set is stored in `requirements-main-lock.txt`. The expanded real-data and affine study uses Python 3.13.7, Torch 2.14.0 with CPU support, GPyTorch 1.15.2, NumPy 2.4.6, pandas 2.3.3, and SciPy 1.16.3. Its exact package set is stored in `requirements-real-data-lock.txt`. The cross-package controls use Python 3.13.7 and the exact package versions in `requirements-independent-controls-lock.txt`. Compression remains in float32 because that is how the released source works. Score calculations use float64 after the source returns its nodes and weights.

The main robustness run completes 24,000 source compressions and 192 unchanged public score checks. The expanded validation adds 44,324 real forecast cases, seed and budget sensitivity checks, equal-budget comparisons, a rolling ranking study, precipitation and bike-demand affine tests, and runtime comparisons. The cross-package module adds 160,000 recorded affine comparisons across equal and unequal weight interfaces. The history module clones the public repository, reads every commit, and stores only commit and blob identifiers. The release includes the scripts, protocols, public data files, JSON results, tables, and figures needed to inspect these claims. It does not redistribute the upstream source tree or the UCI archive. Instead, the scripts fetch the named sources and verify their hashes. `DATA_SOURCES.md` records the origin of each dataset.

A package-level script, `run_all.py`, rebuilds the manuscript, runs four unit tests for the affine wrapper, verifies the stored weather, bike-demand, cross-package, and source-history results, and writes `SHA256SUMS.txt`. The public package is at github.com/jopoku16/crps-affine-audit. The complete submission release is archived at doi:10.5281/zenodo.23302078. The version-independent DOI is doi:10.5281/zenodo.23288535. `CITATION.cff` and `.zenodo.json` provide release metadata.

The documentary audit also records the source commit, a clean source diff, the publication PDF hash, the accessible repository history, and the exact objects used in the map in Table 1. The main simulation output stores per-case seeds, source-call diagnostics, support overlap, weight changes, score changes, reference values, and dataset-level error summaries. The failed cell and its later float64 diagnosis remain separate records. This keeps the planned verdict from being overwritten by a post-run explanation.

10 Conclusion

The released code contains a clear mismatch between unequal compression weights and an equal-weight correction. The resulting translation error has a simple exact formula. Direct calls to the unchanged code agree with that formula. The same target code appears in every public commit that contains the routine. Full-workflow tests show that the problem can change model rankings in the synthetic settings studied here. Temperature, precipitation, and bike-demand data show that the problem is not tied to one variable, one domain, or one kind of unit change.

The affine metamorphic test gives a small and reusable diagnostic for CRPS software. The public package interfaces pass all 120,000 comparisons assigned to them. The two local references also pass their 40,000 comparisons. ASW-CRPS gives a unit-stable way to keep the released compression workflow. Its exact positive-affine law is the main general result beyond the source audit. It is not a new proper score, and it does not prove unbiasedness after adaptive compression. For one-dimensional ensembles, exact sorted fair CRPS is both simpler and much faster in our benchmark. The paper therefore supports a precise correction and a practical diagnostic. It does not support a claim about every published experiment or every forecasting system.

References

[1] J. E. Matheson and R. L. Winkler. Scoring Rules for Continuous Probability Distributions. *Management Science*, 22(10):1087 to 1096, 1976. doi:10.1287/mnsc.22.10.1087.

[2] T. Gneiting and A. E. Raftery. Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association*, 102(477):359 to 378, 2007. doi:10.1198/016214506000001437.

[3] C. A. T. Ferro. Fair scores for ensemble forecasts. *Quarterly Journal of the Royal Meteorological Society*, 2014. doi:10.1002/qj.2270.

[4] M. Zamo and P. Naveau. Estimation of the Continuous Ranked Probability Score with Limited Information and Applications to Ensemble Weather Forecasts. *Mathematical Geosciences*, 2018. doi:10.1007/s11004-017-9709-7.

[5] M. Adachi, M. Fujisawa, and M. A. Osborne. Fixing the Pitfalls of Probabilistic Time Series Forecasting Evaluation by Kernel Quadrature. *Proceedings of the First International Conference on Probabilistic Numerics*, PMLR 271:1 to 11, 2025. Publisher page.

[6] D. M. Kelen, A. Jung, P. Kersch, and A. A. Benczur. Distribution Free Data Uncertainty for Neural Network Regression. *ICLR*, 2025. Conference paper.

[7] Z. Ben Bouallegue, C. A. T. Ferro, M. Leutbecher, and D. S. Richardson. Predictive verification for the design of partially exchangeable multi model ensembles. *Tellus A*, 72(1):1 to 12, 2020. doi:10.1080/16000870.2019.1697165.

[8] S. Hayakawa, H. Oberhauser, and T. Lyons. Positively Weighted Kernel Quadrature via Subsampling. *Advances in Neural Information Processing Systems*, 2022. arXiv:2107.09597.

[9] J. W. Messner. `ensemblepp`: Ensemble Postprocessing Data Sets, version 1.0-0. CRAN, 2019. Package page.

[10] H. Fanaee-T. Bike Sharing [Dataset]. UCI Machine Learning Repository, 2013. doi:10.24432/C5W894.

[11] The Climate Corporation. `properscoring`: Proper scoring rules in Python, version 0.1. Project repository.

[12] F. Zanetta and S. Allen. `scoringrules`: A Python library for probabilistic forecast evaluation, version 0.11.0, 2024. Project repository.

[13] N. Loveday and collaborators. `scores`: Forecast verification in Python, version 2.7.0. Project repository.

[14] R. Bell and collaborators. `xskillscore`: Metrics for verifying forecasts, version 0.0.29. Project repository.

School of Mathematical and Statistical Sciences, University of Texas Rio Grande Valley, Edinburg, Texas 78539, USA
Email addresses: `hurcheson.ofori01@utrgv.edu`, `opokujeffery5@gmail.com`